ACTIVIDAD 1

UNIDAD 1

Jonathan Alfonso Tobon Celemin

Universidad de Cartagena

Ingeniería de Software

Estructura de Datos

John Carlos Arrieta Arrieta

Cartagena de Indias, Abril 2026

LINK REPOSITORIO GITHUB:

https://github.com/JTobon16/Actividad1_EstructuraDeDatos.git

LINK VIDEO YOUTUBE:

https://youtu.be/sQeDxvaRe7c

3. Record o Struct

Fundamentos:

- Concepto y utilidad (almacenar datos relacionados).
- Diferencias entre un registro/struct y un objeto.
- Tipado y acceso a campos.

Actividades:

1. **Declaración**
   - Definir un struct o record que contenga: nombre, edad, promedio.

2. **Inicialización**
   - Crear 3 instancias con datos ficticios en todos los lenguajes dados (Java, C#, JS, TS, Python, Kotlin, Dart, Go, C, C++, PHP).

3. **Recorrido**
   - Guardar las instancias en un arreglo y recorrerlo mostrando los datos.

4. **Modificación**
   - Cambiar el promedio de un estudiante específico.

5. **Comparativa**
   - Documentar las diferencias de implementación en los lenguajes.

JAVA

estudiante.java × Main.java

```java
package struct_record;

public record estudiante(String nombre, int edad, double promedio) { 9 usages Jonathan Tobón
}
```

- ✓ Cree el public record en java

Project

- Actividad1 C:\Users\altoc\IdeaProjects\Actividad1_EstructuraDeDatos\J
  - .idea
  - out
  - src
    - ejercicio26
      - asiento
      - main
      - sala
    - objetos
      - estudiante
      - Main
    - struct_record
      - estudiante
      - Main
  - .gitignore
  - Actividad1.iml
- External Libraries
- Scratches and Consoles

estudiante.java Main.java ×

```java
package struct_record;

public class Main {  Jonathan Tobón
    public static void main(String[] args) {  Jonathan Tobón

        // creamos los estudiantes
        estudiante e1 = new estudiante( nombre: "Jonathan",  edad: 20,  promedio: 4.5);
        estudiante e2 = new estudiante( nombre: "Jinx",  edad: 22,  promedio: 3.8);
        estudiante e3 = new estudiante( nombre: "Andres",  edad: 19,  promedio: 4.2);

        // aqui guardamos el arreglo
        estudiante[] estudiantes = {e1, e2, e3};

        System.out.println("Listado de estudiantes: ");

        // hacemos el recorrido con el for
        for (estudiante e : estudiantes) {
            System.out.println(
                    "Nombre: " + e.nombre() +
                            " Edad: " + e.edad() +
                            " Promedio: " + e.promedio()
            );
        }

        //hacemos la modificacion (record = inmutable)
        e2 = new estudiante( nombre: "Ana",  edad: 22,  promedio: 4.0);
```

- ✓ Hice el ejecutable con la información que quería que tuviera de acuerdo a los parámetros estrablecidos

PYTHON

```python
from dataclasses import dataclass

@dataclass  3 usages  Jonathan Tobón

class Estudiante:
    nombre: str
    edad: int
    promedio: float

    # === Bloque de ejecución propio ===
if __name__ == "__main__":
        r1 = Estudiante("Jonathan", 20, 4.5)
        r2 = Estudiante("Jinx", 22, 3.8)
        r3 = Estudiante("Carla", 19, 4.2)
        lista_r = [r1, r2, r3]

        print(" Información inicial (record) ")
        for e in lista_r:
            print(e)

        # Modificación
        r2.promedio = 4.1
        print("\n Despues de modificar promedio de Jinx ")
        for e in lista_r:
            print(e)
```

✓ En Python si se usa un tipo de record con el uso de dataclass

C#

```csharp
namespace Actividad1.struct_record
{
    5 references
    public struct EstudianteStruct
    {
        public string Nombre;
        public int Edad;
        public double Promedio;

        3 references
        public EstudianteStruct(string nombre, int edad, double promedio)
        {
            Nombre = nombre;
            Edad = edad;
            Promedio = promedio;
        }

        2 references
        public void Mostrar()
        {
            Console.WriteLine($"Nombre: {Nombre}, Edad: {Edad}, Promedio: {Promedio}");
        }
    }

    0 references
    class ProgramStruct
    {
        0 references
        static void Main(string[] args)
        {
            var s1 = new EstudianteStruct("Ana", 20, 4.5);
            var s2 = new EstudianteStruct("Luis", 22, 3.8);
            var s3 = new EstudianteStruct("Carla", 19, 4.2);

            var lista = new EstudianteStruct[] { s1, s2, s3 };

            Console.WriteLine("=== Información inicial (struct) ===");
            foreach (var e in lista)
                e.Mostrar();

            // Modificación
            lista[1].Promedio = 4.1;

            Console.WriteLine("\n=== Después de modificar promedio de Luis ===");
            foreach (var e in lista)
                e.Mostrar();
        }
```

✓ En C# use un tipo Struct tal cual con los parámetros

EstudianteStruct.cs | EstudianteRecord.cs*

Actividad1 | Actividad1.struct_record.ProgramRecord | Main(string[] args)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Actividad1.struct_record
{
    //  Record
    4 references
    public record EstudianteRecord(string Nombre, int Edad, double Promedio);

    // Bloque principal para ejecutar este record
    0 references
    class ProgramRecord
    {
        0 references
        static void Main(string[] args)
        {
            // Inicialización
            var r1 = new EstudianteRecord("Ana", 20, 4.5);
            var r2 = new EstudianteRecord("Luis", 22, 3.8);
            var r3 = new EstudianteRecord("Carla", 19, 4.2);

            var listaR = new List<EstudianteRecord> { r1, r2, r3 };

            Console.WriteLine("=== Información inicial (record) ===");
            foreach (var e in listaR)
                Console.WriteLine(e);

            // Modificacion: con records se crea un nuevo objeto
            r2 = r2 with { Promedio = 4.1 };
            listaR[1] = r2;

            Console.WriteLine("\n=== Después de modificar promedio de Luis ===");
            foreach (var e in listaR)
                Console.WriteLine(e);
        }
```

✓ En C# use un tipo Record tal cual con los parámetros

4. Objetos (Clases e instancias)

Actividades:

1. **Declaración**
   - Definir una clase Estudiante con: nombre, edad, promedio y un método mostrarInfo.

2. **Inicialización**
   - Crear 3 instancias y almacenarlas en un arreglo/lista.

3. **Recorrido**
   - Llamar al método mostrarInfo para cada objeto.

4. **Modificación**
   - Cambiar el promedio de un estudiante mediante un método setPromedio.

5. **Comparativa**
   - Explicar la diferencia entre esta clase y un struct o record.

<u>JAVA</u>

```java
public class estudiante {  9 usages  Jonathan Tobón
    public estudiante(String nombre, int edad, double promedio) {  3 usages  Jonathan Tobón

        this.nombre = nombre;
        this.edad = edad;
        this.promedio = promedio;
    }

    //metodo que muestre la info
    public void mostrarInfo(){  2 usages  Jonathan Tobón

        System.out.println("Nombre: " + nombre +", Edad: " + edad+ ", Promedio "+ promedio);
    }

    //setter para que cambie el promedio

    public void setpromedio(double nuevoPromedio) {  1 usage  Jonathan Tobón
        this.promedio = nuevoPromedio;
    }

}
```

✓ Cree mi objeto y le asigne su método de mostrar y un setter para el cambio de promedio

```java
package objetos;

public class Main {  Jonathan Tobón
    public static void main(String[] args) {  Jonathan Tobón
        estudiante e1 = new estudiante( nombre: "Jonathan",  edad: 26,  promedio: 4.6);
        estudiante e2 = new estudiante( nombre: "Jinx",  edad: 21,  promedio: 3.5);
        estudiante e3 = new estudiante( nombre: "Jose",  edad: 19,  promedio: 4.0);

        estudiante[] lista = {e1, e2, e3};

        System.out.println(" Listado de Estudiantes: ");
        for (estudiante e : lista) {
            e.mostrarInfo();
        }

        // cambiamos promedio de un estudiante (e2)
        e2.setpromedio(4.1);

        System.out.println("\n Despues de la modificacion de Jinx  ");
        for (estudiante e : lista) {
            e.mostrarInfo();
        }
    }
}
```

✓ Cree el main que me ejecutara el código. Llamamos el método y el set.

PYTHON

```python
# aqui creamos la clase tradicional
class Estudiante:
    def __init__(self, nombre, edad, promedio):
        self.nombre = nombre
        self.edad = edad
        self.promedio = promedio

    def mostrar_info(self):
        print(f"nombre:{self.nombre}, edad:{self.edad}, promedio:{self.promedio}")

    def set_promedio(self,nuevo_promedio):
        self.promedio = nuevo_promedio

# aqui estara la ejecucion del programa
if __name__ == "__main__":
    r1 = Estudiante("Jonathan", 20, 4.5)
    r2 = Estudiante("Jinx", 22, 3.8)
    r3 = Estudiante("Carla", 19, 4.2)

    lista = [r1, r2, r3]

    print("Informacion inicial de la clase")
    for e in lista:
        e.mostrar_info()

#aqui hicimos la modificacion del promedio
    r2.set_promedio(5)
    print("\n Despues de la modificacion de Jinx")
    for e in lista:
        e.mostrar_info()
```

✓ Cree mi clase tradición con su objeto definido, y la respectiva ejecución del programa

✓ 5. Diferencias entre Struct/Record y Objetos

Actividad teórico-práctica:

- Los estudiantes crearán una tabla comparativa con:
  - Definición
  - Mutabilidad
  - Tipado
  - Uso en memoria (stack/heap)
  - Ejemplo en un lenguaje estático y uno dinámico

Concepto	Struct	Record	Objeto (Clase)
Definición	Tipo de dato que agrupa variables simples	Tipo de dato para representar información inmutable	Entidad con atributos y comportamiento (métodos)
Mutabilidad	Generalmente mutable	Generalmente inmutable	Mutable
Tipado	Fuerte y estático	Fuerte y estático	Fuerte y estático
Uso en memoria	Stack (más rápido, ligero)	Heap (mas grande y lento)	Heap (mas grande y lento)

Uso principal	Datos pequeños y simples (coordenadas, valores)	Representar datos sin modificar (ej: registros)	Modelar entidades completas con lógica
Complejidad	Baja	Media	Alta

ACTIVIDAD PRÁCTICA:
Modificar el ejercicio asignado en la actividad Protocolo Individual Unidad 1 (ver lista de asignación de ejercicios) y aplicar los conceptos de Objetos, Struct y Récord como elementos (items) guardados en los arreglos o matrices. (Java) opcional y mejor calificación si lo hace en otro de los lenguajes indicados en esta actividad.

<u>EJERCICIO 26:</u> Asignación de asiento en sala de cine

- **JAVA**

asiento.java × sala.java main.java

```java
package ejercicio26;


public class asiento {  5 usages  Jonathan Tobón
    //aqui defini el objeto del ejercicio, aqui no use matrices sino directamente un objeto dentro
    private int fila;  1 usage
    private int columna;  1 usage
    private boolean reservado;  5 usages
    //hice el contructor de nuestro objeto
    public asiento(int fila, int columna) {  1 usage  Jonathan Tobón
        this.fila = fila;
        this.columna = columna;
        this.reservado = false;
    }
    //cree mis dos metodos necesarios, el reservar y liberar
    public void reservar() {  1 usage  Jonathan Tobón
        reservado = true;
    }

    public void liberar() {  1 usage  Jonathan Tobón
        reservado = false;
    }

    public boolean isReservado() {  no usages  Jonathan Tobón
        return reservado;
    }
    //aqui nuestro mostrar, la parte visual al ejecutar
    public String mostrar() {  1 usage  Jonathan Tobón
        return reservado ? "[X]" : "[ ]";
    }
```

asiento.java sala.java × main.java

```java
package ejercicio26;

public class sala {  2 usages  Jonathan Tobón
    //aqui creo mi matris del objeto
    private asiento[][] asientos;  5 usages
    //constructor, creo la matriz vacia
    public sala(int filas, int columnas) {  1 usage  Jonathan Tobón
        asientos = new asiento[filas][columnas];

        //el respectivo recorrido de la matriz
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                asientos[i][j] = new asiento(i, j);
            }
        }
    }
    // el mostrar sala, use el for each
    public void mostrarSala() {  3 usages  Jonathan Tobón
        for (asiento[] fila : asientos) {
            for (asiento a : fila) {
                System.out.print(a.mostrar());
            }
            System.out.println();
        }
    }
    //el reservar y liberar, llamo los metodos creado en mi objeto
    public void reservar(int fila, int columna) {  1 usage  Jonathan Tobón
        asientos[fila][columna].reservar();
    }
```

Project

- Actividad1 C:\Users\altoc\IdeaProjects\Actividad1_EstructuraDeDatos\J
  - .idea
  - out
  - src
    - ejercicio26
      - asiento
      - main
      - sala
    - objetos
    - struct_record
  - .gitignore
  - Actividad1.iml
- External Libraries
- Scratches and Consoles

asiento.java sala.java main.java ×

```java
package ejercicio26;

public class main {  Jonathan Tobón
    public static void main(String[] args) {  Jonathan Tobón

        sala sala = new sala( filas: 5,  columnas: 5);

        System.out.println("Sala de Cine");

        sala.mostrarSala();

        System.out.println("\nReservando asiento (3,3)\n");
        sala.reservar( fila: 2,  columna: 2);

        sala.mostrarSala();

        System.out.println("\nLiberando asiento (3,3)\n");
        sala.liberar( fila: 2,  columna: 2);

        sala.mostrarSala();
    }
```

- **PYTHON**

```python
#con esto converti la clase en algo tipo record
from dataclasses import dataclass

@dataclass  1 usage  Jonathan Tobón
#se crea la clase ya con los constructores
class Asiento:
    fila: int
    columna: int
    reservado: bool = False

    #metodos necesarios
    def reservar(self):  1 usage  Jonathan Tobón
        self.reservado = True

    def liberar(self):  1 usage  Jonathan Tobón
        self.reservado = False

    def mostrar(self):  1 usage  Jonathan Tobón
        return "[X]" if self.reservado else "[ ]"

#creo la clase Sala con las matrices y parametros necesarios
class Sala:  1 usage  Jonathan Tobón
    def __init__(self, filas, columnas):  Jonathan Tobón
        self.asientos = [[Asiento(i, j) for j in range(columnas)] for i in range(filas)]
    #hago el recorrido con el for para que me muestre la matriz
    def mostrar(self):  3 usages  Jonathan Tobón
        for fila in self.asientos:
            print("".join(a.mostrar() for a in fila))
    #la reserva
    def reservar(self, f, c):  1 usage  Jonathan Tobón
        self.asientos[f][c].reservar()
    #la liberacion
```

- **C#**

```csharp
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Actividad1.ejercicio26
{

    public struct Posicion
    {
        public int Fila;
        public int Columna;


        public Posicion(int fila, int columna)
        {
            Fila = fila;
            Columna = columna;
        }
    }
    //se usa struct porque es mas ligero al guardar datos
}

```

```csharp
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Actividad1.ejercicio26
{
    // aqui use struct dentro de la clase usando el Pos
    4 references
    public class Asiento
    {
        1 reference
        public Posicion Pos { get; set; }

        //guarda si esta ocupado
        6 references
        public bool Reservado { get; set; }

        1 reference
        public Asiento(int fila, int columna)
        {
            //este es el constructor
            Pos = new Posicion(fila, columna);
            Reservado = false;
        }
        // los metodos
        0 references
        public void Reservar()
        {
            Reservado = true;
        }

        0 references
        public void Liberar()
        {
            Reservado = false;
        }

        1 reference
        public string Mostrar()
        {
            return Reservado ? "[X]" : "[ ]";
        }
    }
}
```

Sala.cs | **Reserva.cs** | Program.cs | Posicion.cs | Asiento.cs

Actividad1 | Actividad1.ejercicio26.Reserva

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Actividad1.ejercicio26
{
    //creo el objeto simple, al ser record es inmutable, es equivalente al dataclass en python
    0 references
    public record Reserva(int Fila, int Columna);
}
```

Sala.cs | Reserva.cs | Program.cs | Posicion.cs | Asiento.cs

Actividad1 | Actividad1.ejercicio26.Sala

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Syste
    ( ) namespace System

namespace Actividad1.ejercicio26
{
    3 references
    public class Sala
    {
        //aqui a diferencia de java se usa la matriz de objeto
        private Asiento[,] asientos;

        1 reference
        public Sala(int filas, int columnas)
        {
            asientos = new Asiento[filas, columnas];

            for (int i = 0; i < filas; i++)
            {
                for (int j = 0; j < columnas; j++)
                {
                    asientos[i, j] = new Asiento(i, j);
                }
            }
        }
        3 references
        public void Mostrar()
        {
            for (int i = 0; i < asientos.GetLength(0); i++)
            {
                for (int j = 0; j < asientos.GetLength(1); j++)
                {
                    Console.Write(asientos[i, j].Mostrar());
                }
                Console.WriteLine();
            }
        }
        1 reference
        public void Reservar(int f, int c)
        {
            asientos[f, c].Reservado = true;
        }
        1 reference
        public void Liberar(int f, int c)
```

Sala.cs | Reserva.cs | **Program.cs** | Posicion.cs | Asiento.cs

Actividad1 | Actividad1.ejercicio26.Program | Main()

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Actividad1.ejercicio26
{

    class Program
    {

        static void Main()
        {
            Sala sala = new Sala(5, 5);

            Console.WriteLine("Sala de Cine");
            sala.Mostrar();

            Console.WriteLine("\nReservando asiento (3,3)\n");
            sala.Reservar(2, 2);

            sala.Mostrar();

            Console.WriteLine("\nLiberando asiento (3,3)\n");
            sala.Liberar(2, 2);

            sala.Mostrar();
        }
    }
}
```

Solution Explorer

Search Solution Explorer (Ctrl+`)

- Solution 'Actividad1' (1 of 1 project)
  - **Actividad1**
    - Dependencies
    - Properties
    - ejercicio26
      - Asiento.cs
      - Posicion.cs
      - Program.cs
      - Reserva.cs
      - Sala.cs
    - objetos
    - struct_record
    - .gitignore.txt
    - Program.cs